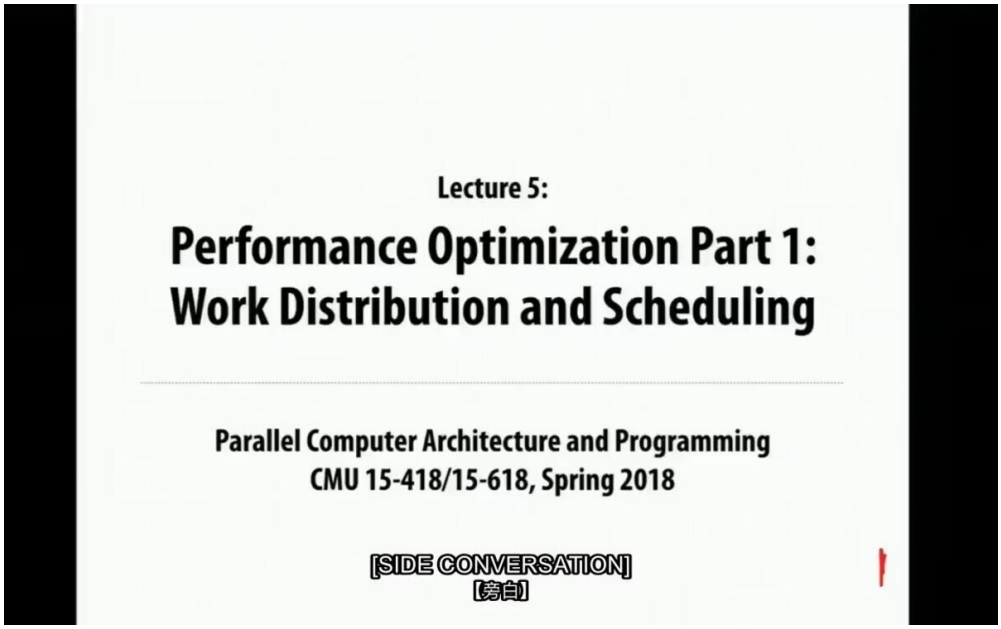
课程笔记

工作分配与调度：均衡与开销之间的取舍

CMU 15-418/618 并行计算机架构与编程 · Lecture 5（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 24 秒
视频链接：`https://www.bilibili.com/video/BV1bp4y1G7tJ/`

目录

1 性能优化的三个目标 **3**

1.1 迭代是方法的本质 3

1.2 操作建议：先写最简单的版本 3

1.3 三个必须同时盯住的目标 3

1.4 负载不均的代价有多大 4

1.5 本章小结 4

2 静态分配：预先决定，零运行期开销 **5**

2.1 做法与它的最大优点 5

2.2 适用条件：任务时间可预测 5

2.3 本章小结 6

3 半静态分配：在两者之间取一个折中 **6**

3.1 为什么需要中间方案 7

3.2 定期重测，期间静态 7

3.3 本章小结 8

4 动态分配：让线程按需取活 **8**

4.1 最简单的实现 8

4.2 工作不必是循环迭代 9

4.3 本章小结 9

5 细粒度动态分配的代价与粒度旋钮 **9**

5.1 锁竞争吃掉了时间 9

5.2 一次取多个 10

5.3 粒度过大的代价 11

5.4 收尾陷阱：最后派出的长任务 11

5.5 一个理想但需要知识的策略 12

5.6 本章小结 12

6 分布式队列与 work stealing **12**

6.1 每线程一个队列 13

6.2 队列耗尽的必然性 13

6.3 窃取的三个设计要点 14

6.4 收尾阶段的集中争用 15

6.5 本章小结 15

7 Cilk 的 fork-join：抽象如何决定调度细节 **16**

7.1 分治与 fork-join 模式 16

7.2 spawn 与 sync 17

7.3 为什么不能每个 spawn 开一个 pthread 18

7.4 child 与 continuation 19

7.5 该先做 child 还是先做 continuation 21

7.6 Cilk 的真实选择 22
7.7 本章小结 23

8 总结与延伸 **23**

1 性能优化的三个目标

这一讲从”能跑”转向”跑得快”。讲者开宗明义：并行编程是高度迭代的过程，第一次几乎不可能做对，第二次甚至第三次也未必。因此真正的方法不是一次设计到位，而是测量、从测量中学习、再改进。

1.1 迭代是方法的本质

讲者把这条作为全讲的前提。学生在做作业时会亲身体会：必须测量性能，从测量里读出信息，再用这些信息改进。这也直接推出了一条建议。

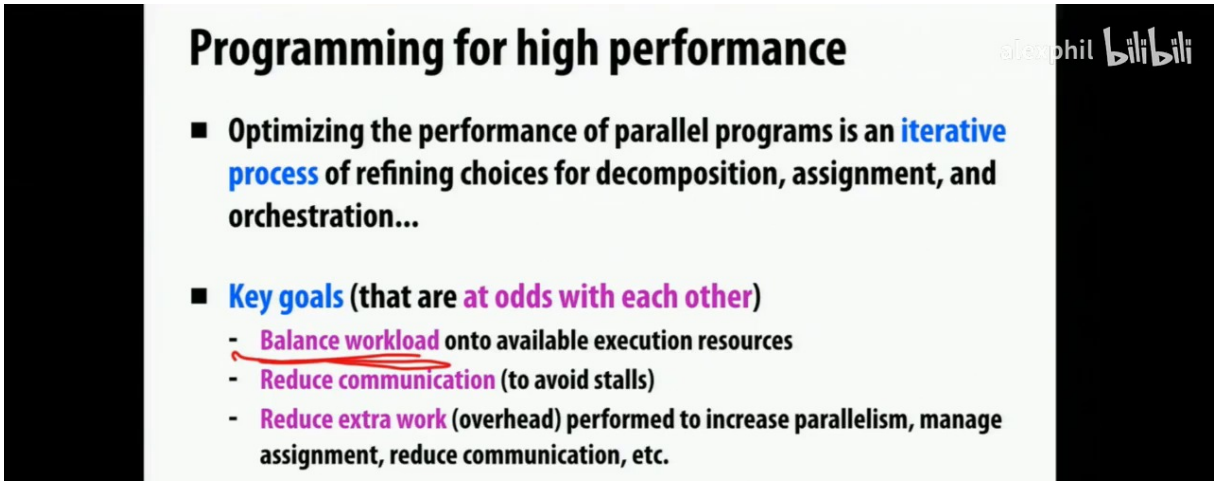


图 1: 高性能编程：优化并行程序是一个迭代过程[1]

1.2 操作建议：先写最简单的版本

讲者的建议很具体：先实现能工作的最简单方案。原因有两条。简单的实现你能更快写完，因此更早拿到测量数据；而当被测量的对象本身简单时，你也更容易理解测量结果的含义。有了这个基准，再去做复杂优化才有一个可靠的起点。

简单版本的价值是提供一个可解释的基准

复杂方案第一次尝试通常不会成功，此时你无法判断失败来自设计缺陷还是实现错误。简单版本给出的数字虽然不好看，但它干净、可复现、可解释，后续每次改动都能与它对比。这是一个测量基准，不是临时凑合的代码。

1.3 三个必须同时盯住的目标

讲者把今天要处理的问题归纳为三项，它们之间存在张力。

第一项是均衡负载，让所有线程有相同的工作量。有时很容易，有时很难，这是今天的主线。

第二项是减少通信，因为通信昂贵。

第三项是控制软件开销。为了优化而引入的任何聪明做法，往往意味着运行更多指令，这份开销同样会拖慢程序。

[1] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

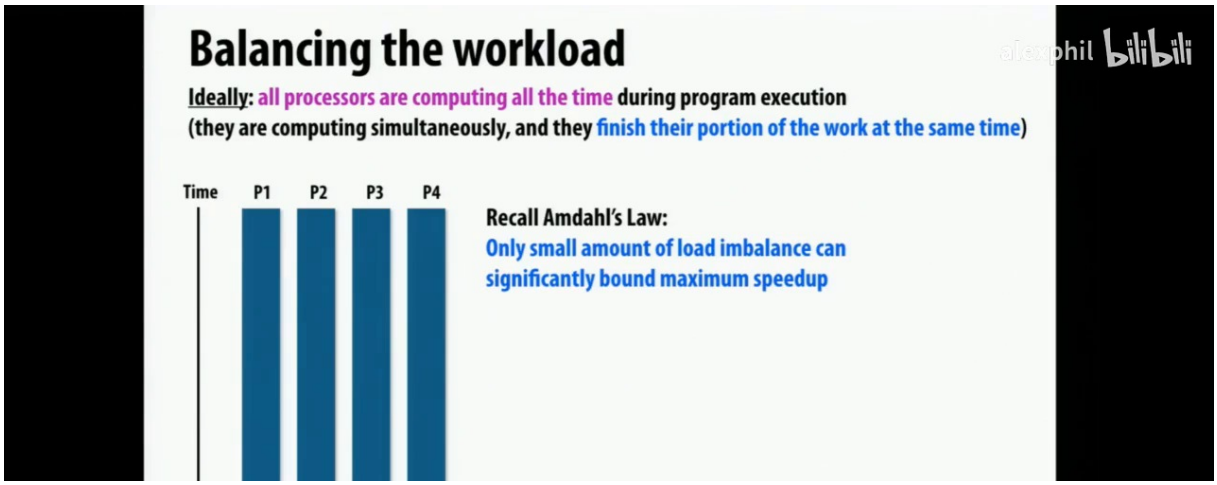


图 2: 负载均衡：理想情形是所有处理器在整个执行期间都在计算[2]

1.4 负载不均的代价有多大

讲者用第一堂课的演示作为参照，然后给出一个更精确的说明：即使把工作量分得几乎完全正确，只差一点也会很贵。

他的例子是四个处理器，其中前三个的工作量完全相同，第四个只多出一小部分，四者的差距在 20% 以内。看起来分得不错。但在那第四个多花的时间里，其余三个处理器无事可做。

并行程序的完成时间由最慢任务决定：

$$T_{\text{parallel}} = \max_{i=1\ldots P} T_i, \qquad E_{\text{balance}} = \frac{\sum_i T_i}{P \cdot \max_i T_i}$$

- T_i：第 i 个处理器分到的工作总耗时；
- P：处理器数量；
- T_{parallel}：整体完成时间，等于最慢者的耗时；
- E_{balance}：负载均衡效率，取值越接近 1 越好。

若四者中有三个相同、另一个多出两成，则 E_{balance} 约为 0.95，看似尚可，但那 5% 全部落在收尾阶段，成为无法并行的等待。

负载不均的代价由最慢者决定

并行程序的完成时间取决于最慢的那个处理器，而不是平均值。四者相差 20% 之内看似均衡，实际损失的是整个收尾阶段的并行度。判断均衡程度要看最大工作量与平均工作量之比，不是看方差。

1.5 本章小结

性能优化是迭代过程，起点应是可解释的简单实现。要同时权衡三件事：均衡负载、减少通信、控制软件开销。其中负载不均的代价按最慢者计算，20% 的差距就足以浪费一段收尾时间。

[2] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

这条约束引出核心问题：既然负载不均如此昂贵，那么如何把工作分下去？下一章从最简单的做法开始。

2 静态分配：预先决定，零运行期开销

静态分配是在程序运行前就确定好每个处理器做哪些工作。

2.1 做法与它的最大优点

讲者回顾上一讲的网格求解器：最终版本的代码里用的是块分配，也讨论过交叉分配。无论哪种，分配逻辑都已经写进代码，因此永远按这个方式分给处理器。

所谓”预先确定”并不排斥少量运行期计算。程序仍需要读取输入文件大小、矩阵规模、处理器数量，据此算一遍。但真正的并行工作开始之前，划分已经完全确定。

它的最大优势是运行期开销接近零：既然已经想过怎么分，运行期不需要再做额外的事。

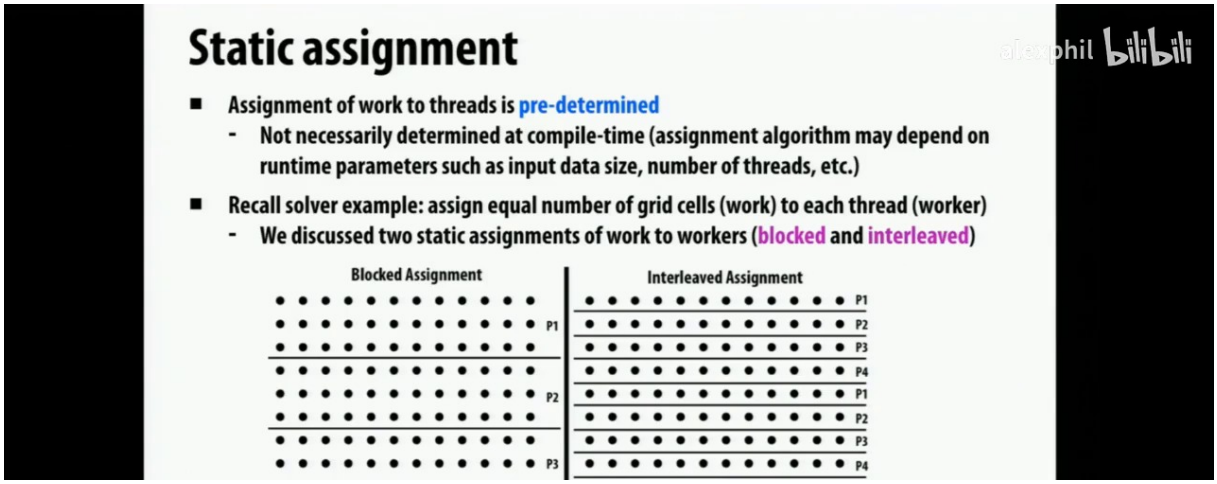


图 3: 静态分配：工作到线程的划分在运行前确定[3]

2.2 适用条件：任务时间可预测

学生很快指出了它的缺点：如果某个任务意外地比预期慢很多，你就只能跟着它。

讲者由此给出适用条件。网格求解器之所以能静态分配，是因为每个网格点的计算完全相同，除了缓存缺失等小扰动之外，各任务耗时非常一致。

更一般地说，需要的是任务运行时间可预测。最容易可预测的情形是它们全都一样。另一种情形是任务大小不同，但与某个参数成比例（例如线性相关），那么虽然大小不一，仍能预测。

[3] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

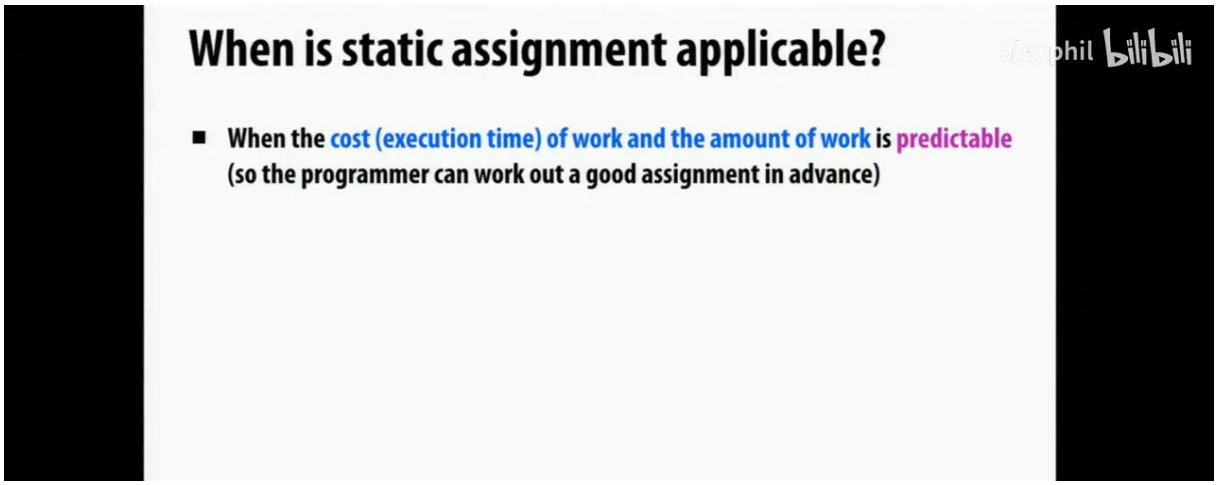


图 4: 静态分配的适用条件：工作开销与工作量可预知[4]

在后一种情形下，做法是把任务打包，使每个处理器分到的任务预期时间之和大致相等。

$$\left| \sum_{k \in S_i} \hat{T}_k - \frac{1}{P} \sum_{k=1}^{M} \hat{T}_k \right| \le \varepsilon$$

- S_i：分给第 i 个处理器的任务集合；
- $\hat{T}_k$：任务 k 的预估耗时；
- M：任务总数；
- ε：可接受的不均衡容差。

打包是个组合优化问题

把大小不一的任务分成 P 组且各组和尽量相等，一般情形下是 NP 难的。实际做法是用贪心（最大任务优先放入当前最轻的组）或按已知的规律直接排序分配。因此静态分配的”均衡”始终是近似的，只是近似误差被压在可接受范围内。

只要预估准确，这个打包就能在运行期零开销的情况下接近均衡。这样打包未必完美，但优点仍然是运行期几乎无开销。讲者的判断是：如果只牺牲一点均衡能换回极低的运行期开销，这笔交易值得做。

2.3 本章小结

静态分配把划分写在代码里，运行期开销接近零，代价是对可预测性的强依赖。任务耗时一致，或至少能按参数预估时，它是最优选择。若做不到这一点，就需要把决策推迟——但推迟也有程度之分。

3 半静态分配：在两者之间取一个折中

讲者特意强调，这一节的内容往往不易想到，但很值得掌握。

[4]视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

3.1 为什么需要中间方案

很多系统按时间步推进模拟。每个任务的耗时可能随时间变化，看起来无法静态划分。但如果这份变化很缓慢，情况就不同了。

星系模拟就是例子。上一讲提过 Barnes-Hut 算法的工作量不均匀：某个星体周围若有很多邻居，它要做大量成对比较；比较孤立的星体则可以把远处星群合并成一个质量块，省下很多计算。所以工作量既不均匀，又随时间变化。

但讲者的关键补充是：星体移动得并不快。

3.2 定期重测，期间静态

于是可以做半静态调度。每隔一段时间，测量一次当前各任务的耗时，据此重新决定划分。然后在此后若干次迭代里，就按这个划分当作静态调度执行，不再重新审视。

重划分周期的两个约束

周期太短，测量与重划分本身的开销占比过高；周期太长，划分依据会过时，负载已经漂移。合理周期取决于负载变化的时间尺度与一次重划分的代价，通常取”重划分开销只占该区间总时间几个百分点”的量级。

等过了一段时间，再回来重新测量、重新划分。

讲者说这类技术在其他物理模拟中同样适用，例如风洞里的飞机模型：物体在动、状态在变，但通常不会剧烈到需要每步重划分。

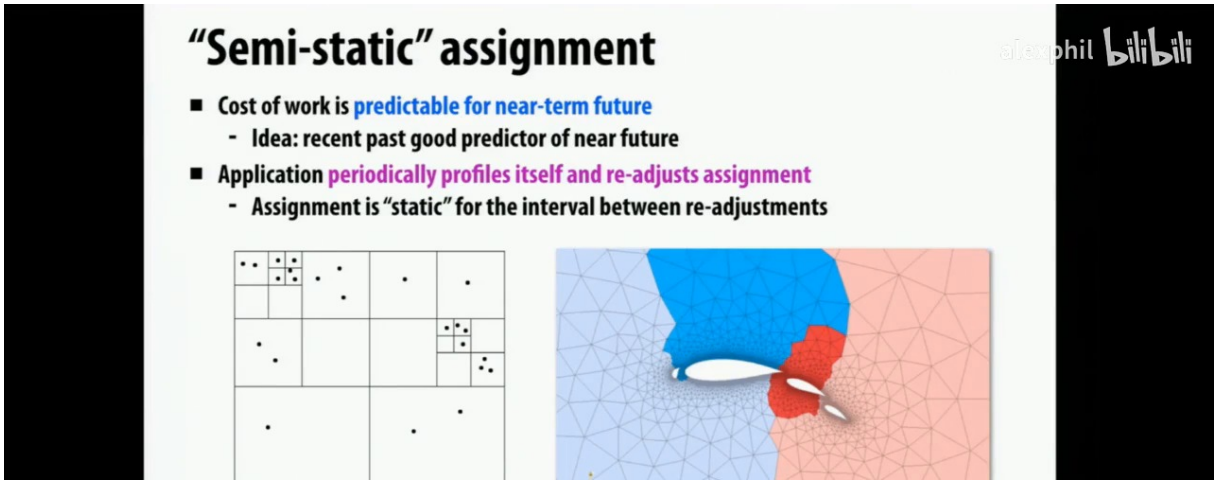


图 5：半静态分配：在近期未来内工作的开销仍可预测[5]

半静态的前提是变化慢于重划分周期

半静态只在负载变化的时间尺度远大于重划分间隔时有效。若网络流量这类负载在毫秒级剧烈波动，周期性重划分反而会一直基于过时的信息，此时纯动态调度更合适。判断依据是负载的自相关时间，而不是数据规模。

[5]视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

3.3 本章小结

半静态把”定期测量加期间静态”组合起来，在负载缓慢变化时用极低的平均开销获得接近动态的均衡质量。它填补了静态与动态之间的空白，也说明分配策略是一条连续谱，而非两种对立选择。

当负载真的不可预测时，就只能把决策做到运行期。下面四章都在处理这种情形的代价。

4 动态分配：让线程按需取活

动态分配在程序运行期间由线程自己决定拿哪份工作。

4.1 最简单的实现

讲者用一段代码说明。假设有一个顺序循环，各次迭代彼此独立，那么每次迭代就是一个任务。可以静态地按块或交叉分配，也可以动态地这样写：维护一个计数器，本质上就是循环下标；线程需要工作时读取这个计数器并自增，因此必须加锁或使用等价的原子指令，否则计数会损坏。

动态分配的均衡以正确性为前提

动态队列只保证工作被分出去，不保证迭代之间真的独立。若循环体存在跨迭代依赖，动态分配的先后顺序会改变结果，且每次运行可能不同。判断能否动态调度，先确认任务独立性，再谈粒度。

各线程拿到一个迭代就去做自己的计算。如果某些迭代远比其他耗时，最终也能大致平衡，因为你只有在准备好时才回去取新工作。

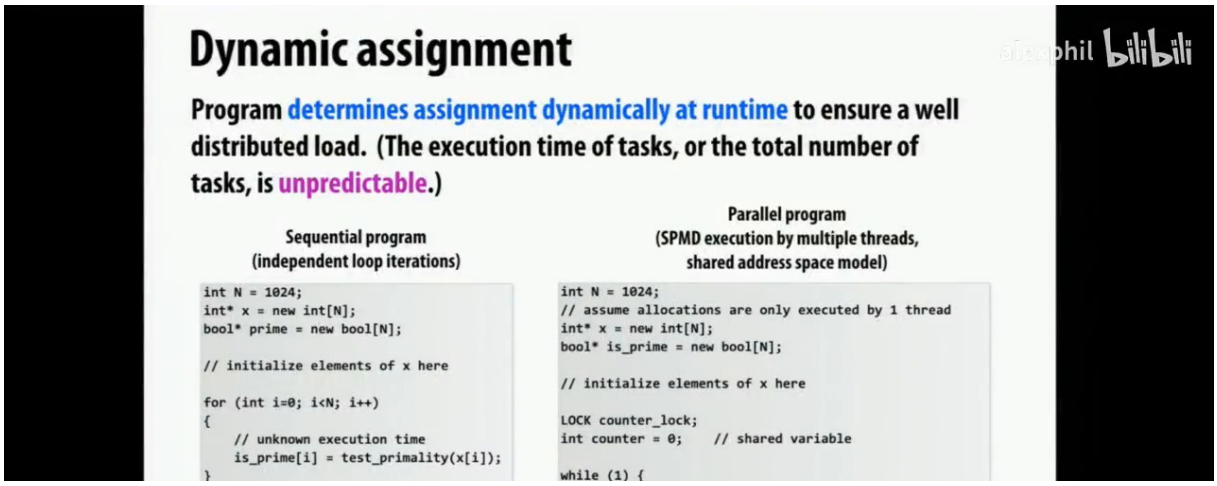


图 6: 动态分配：程序在运行期决定划分以保证负载均衡[6]

动态分配的最大优势是它适用于执行时间不可预测、且波动较大的情形。这正是静态分配失效之处。

[6]视频画面时间区间：00:14:00–00:14:15。若为裁剪图，时间区间仍对应原视频画面。

4.2 工作不必是循环迭代

讲者补充了一点：前面把工作表示成循环迭代，但更一般地，工作可以是任何需要数据结构来描述的东西。星系模拟里的每颗星就是四叉树中的一个节点，我们要访问所有节点。

若要纯动态调度，可以为每个任务建一个小数据结构，全部丢进一个大工作队列。有四个核，就有四个工作线程，谁需要工作就去队列里取。

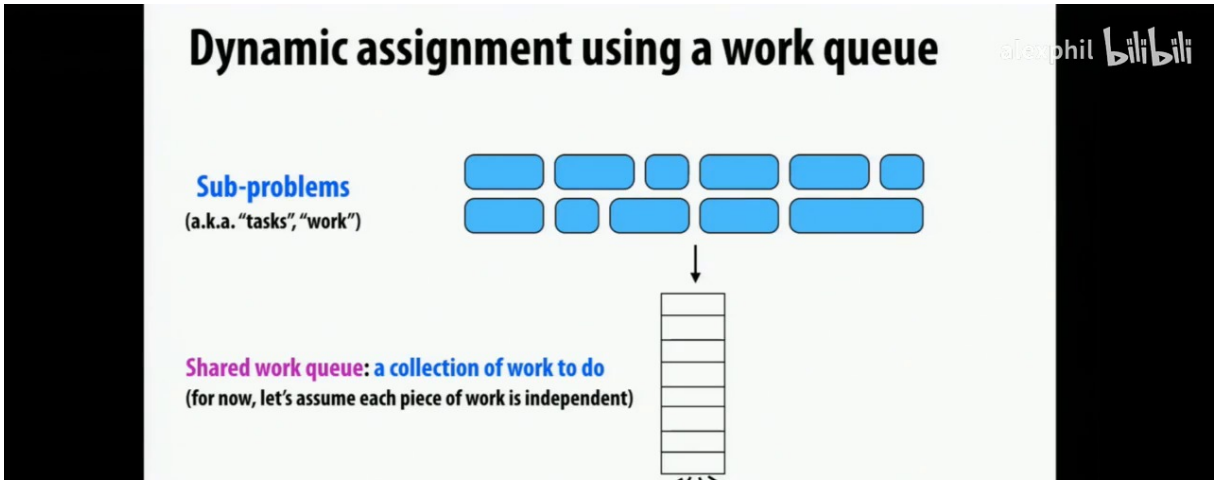


图 7: 用工作队列实现动态分配：所有任务进入共享队列[7]

4.3 本章小结

动态分配用共享计数器或工作队列把划分推迟到运行期，因此能适应不可预测的耗时。它简单、正确，但讲者随即提出一个尖锐问题：这样写的代码，性能如何？

5 细粒度动态分配的代价与粒度旋钮

上一章的代码在功能上没有问题。讲者故意追问：如果它跑得特别慢，你认为原因是什么？

5.1 锁竞争吃掉了时间

答案是加锁。线程频繁去抢那把保护计数器的锁，大量时间花在竞争上而非计算上。这种一次只取一个任务的写法叫细粒度动态分配。

[7]视频画面时间区间：00:16:30–00:16:45。若为裁剪图，时间区间仍对应原视频画面。

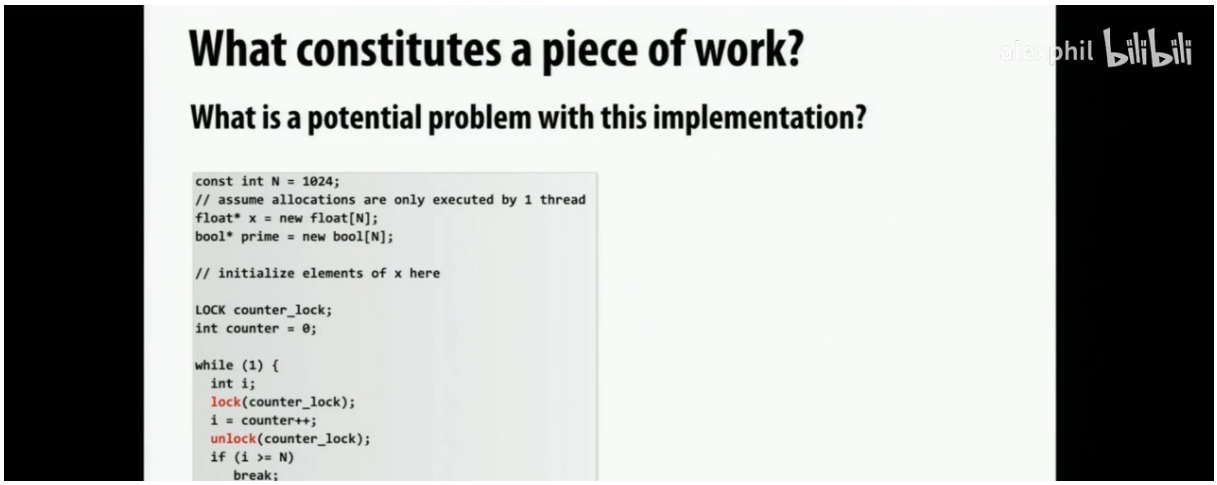


图 8: 什么算一块工作：一次只取一个迭代的写法及其问题[8]

讲者用时间轴把损失画出来：蓝色是真正做有用工作的时间，白色是去队列抢锁、访问计数器的时间。图上白色段极多。他明确指出，白色部分实际上就是 Amdahl 定律里的串行时间，完全不是有效工作时间。

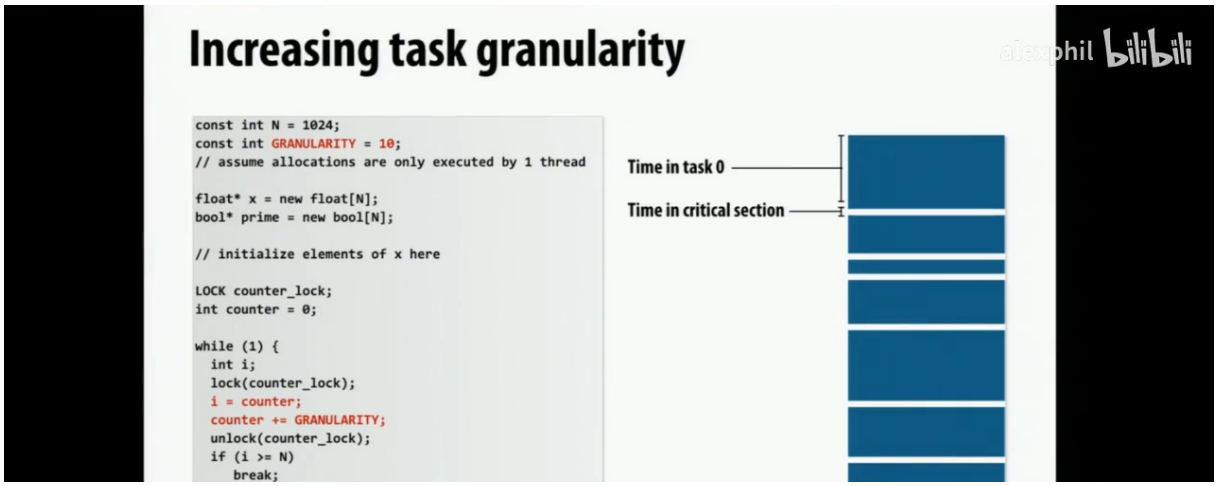


图 9: 增大任务粒度：一次抓取多个迭代来减少取任务次数[9]

5.2 一次取多个

学生提出的办法是：不要只算一个，一次取若干个（比如五个或十个）。线程一次从计数器拿到一段，然后遍历这一整段。

好处是任务变大，回去取新工作的频率下降，白色那部分开销因此减少。

把开销写成显式模型就能看清旋钮的作用：

$$T_{\text{total}} = \frac{W}{P} + \frac{W}{G} \cdot c_{\text{lock}} + \delta_{\text{imbalance}}$$

- W：总工作量；

[8] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[9] 视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

- P：线程数；
- G：每次抓取的任务数，即粒度；
- c_{lock}：一次加锁取任务的代价；
- $\delta_{\text{imbalance}}$：粒度变粗后重新出现的负载不均损失。

G 增大时中间项下降、末项上升，因而存在最优值。

粒度是并行调度最有效的单一旋钮

一次取多少任务，直接决定两项成本的比例。取得少，均衡好但取任务开销占比高；取得多，开销低但可能重新出现负载不均。多数调度器的默认粒度值都是按这个权衡调出来的，调优时优先改这一个参数。

5.3 粒度过大的代价

讲者接着问：这么做有什么坏处？答案是粒度太粗会重新引入负载不均。

所以在光谱的两端都不好——工作太少或任务太大伤均衡，任务太碎伤开销。讲者说，并行编程里很多事情都是这样，中间存在一个合适的位置。

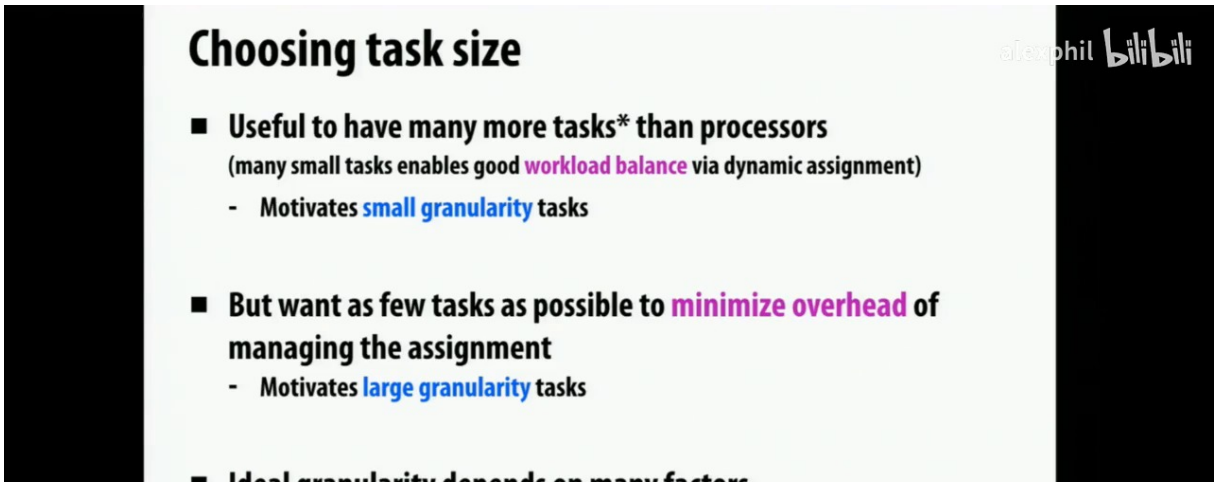


图 10：任务规模的选择：任务数应远多于处理器数[10]

5.4 收尾陷阱：最后派出的长任务

讲者随后展示了一个即使粒度合适也会出问题的情况。任务从左到右进入队列，依次派出，最后派出的那一件恰好在最右边。

如果那件恰好耗时长，就会出现这样的情形：其余处理器很快做完，然后全体坐着等这件大任务。这属于运气不好，但确实会发生。

[10] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。



图 11：更聪明的任务调度：最后才派出的大任务导致全体空等[11]

5.5 一个理想但需要知识的策略

学生提出给任务优先级。讲者认可：如果事先知道这是个大任务，早点派出去比放在最后好得多。若把同一份工作提前派给某个处理器，那个处理器会花较长时间做它，而其他线程能找到别的任务，总体就均衡了。

由此得到一条更深的洞察。在这样一张负载图上，最优调度应当是执行早期用较大的任务，只在接近结束时才需要较小的任务。原因是大任务能压住运行期开销，而这个优势会一直成立到收尾阶段——那时拼图块对不上了，才开始浪费时间。

讲者的说法很形象：开头用大球，结尾用沙粒。这需要事先对哪些任务更大有一些粗略知识；另一个变体是随时间动态调整粒度，例如示例中的粒度 10，可以在接近结束时逐步调小。

收尾阶段的并行度天然衰减

无论分配策略多好，最后总剩少量工作无法再被均分，此时并行度会从 P 掉到 1。这不是调度失误，而是工作量有限的必然结果。改善方向是让最后剩下的任务尽可能小（因此开头可以用大任务），而不是试图消除这个阶段。

5.6 本章小结

细粒度动态分配被锁竞争拖垮，因为每次取任务都要抢锁，这段等待本质上就是 Amdahl 定律中的串行部分。加大粒度能压住这笔开销，但过大会重新引入不均，因此存在最优区间；更精细的做法是让粒度随时间从大到小。

粒度问题解决后还有一处结构性问题：所有线程都在抢同一个队列，这份争用本身也需要消除。这是下一章的内容。

6 分布式队列与 work stealing

即使取任务的频率已经降下来，只要队列真的是一个中央队列，每次访问仍要与其他线程通信、争抢同一把锁。讲者由此引出替代方案。

[11] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

6.1 每线程一个队列

把队列拆开，每个硬件线程一个自己的队列，这叫分布式工作队列。

做法是先按最佳猜测把工作铺到各队列，然后开始运行。此后某线程始终优先访问自己的队列；计算过程中若产生新工作，也放回自己的队列。

这样在大部分执行时间里效果极好：访问自己的队列，没有人与你争用，局部性也很好。

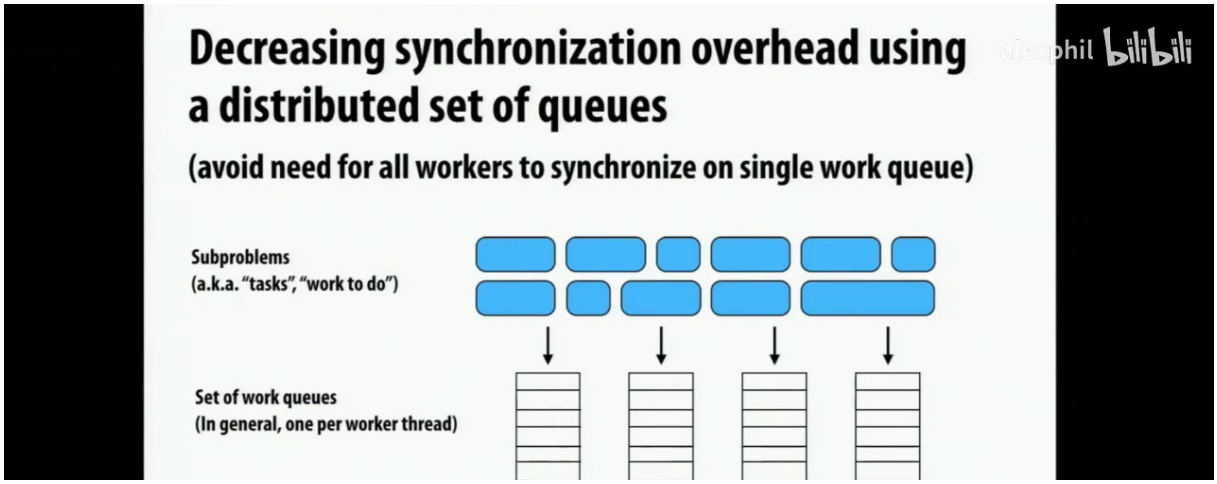


图 12: 用分布式工作队列降低同步开销[12]

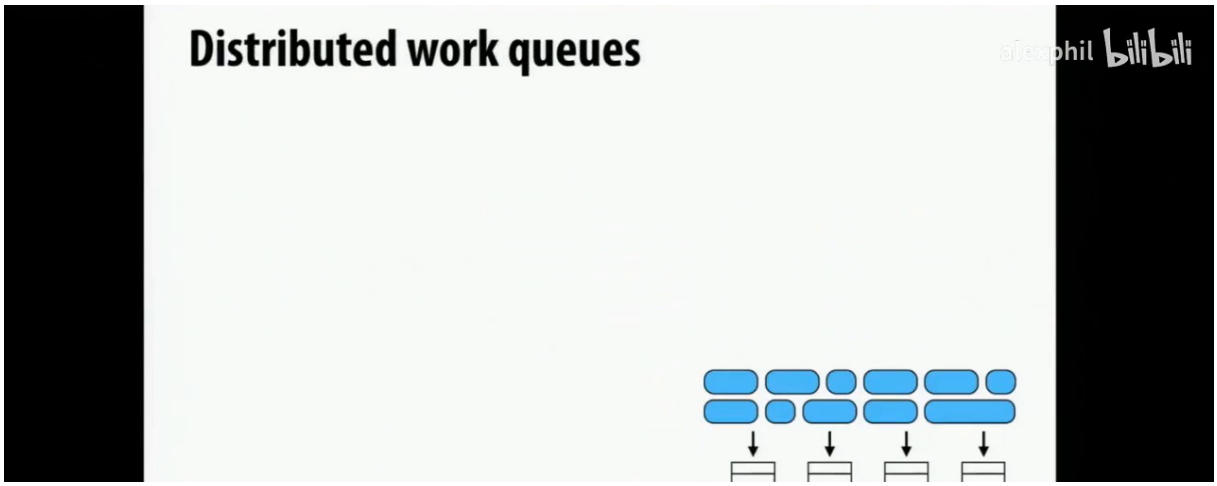


图 13: 分布式工作队列：每个处理器一个队列[13]

6.2 队列耗尽的必然性

问题随之而来：我们之所以用动态调度，正是因为难以预测执行时间并平均切分。既然一开始就分不准，就必然有线程会先把自己的队列做空。

最简单的做法是干等。也可以设一个备用队列，让早完成的线程从那里取工作，争用人数会比中央队列少。

[12] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:31:30–00:31:45。若为裁剪图，时间区间仍对应原视频画面。

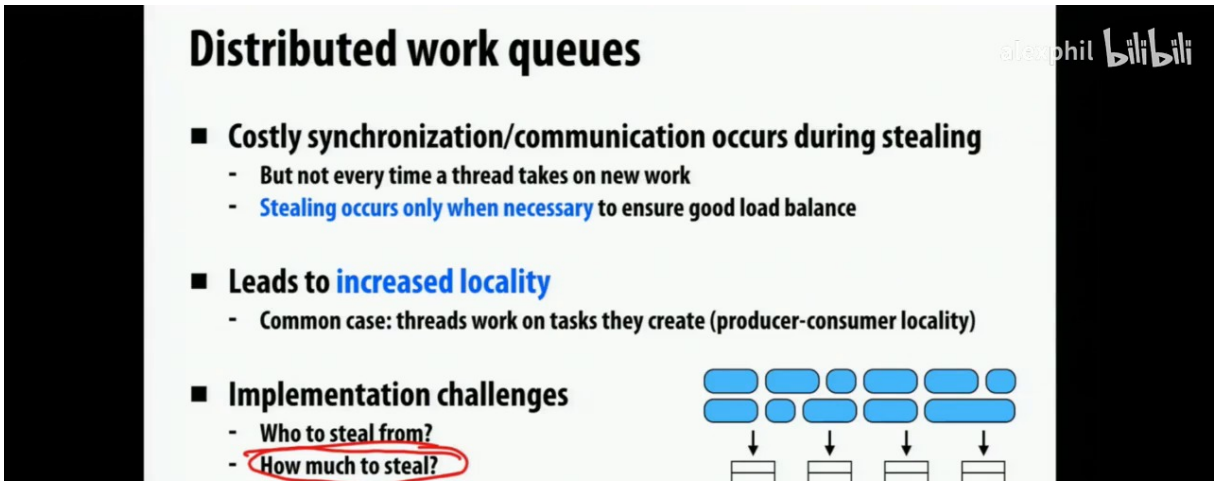


图 14: 分布式队列下的窃取：只在必要时才去别处取工作[14]

6.3 窃取的三个设计要点

讲者给出的主流答案是 work stealing：工作用完时去别的队列偷，这样不必干坐着。

偷工作不是免费的，涉及通信与同步。但好在只有有人没活时才会发生，因此大部分执行期间没有这项开销。讲者提醒仍需小心：不要一进入窃取就突然切换成极慢的模式。

关于从谁那里偷，Cilk 的运行时采用随机选择。讲者解释为什么不能总是从编号小的开始偷：如果所有线程都先去偷 0 号、再偷 1 号，反而会制造新的失衡。

窃取策略必须避免对称

让所有线程按同一顺序去偷（先偷 0 号、再偷 1 号），会在空闲线程之间制造新的热点：所有空闲者同时扑向同一个队列。随机选择或按线程编号做散列，目的都是打破这种对称，使窃取请求均匀分散。

因此要随机，或者用任何能保证大致均匀的方式。

关于偷多少，答案是一次偷多个。只偷一个的话，很可能马上又要再偷一次。

若每次只偷一个任务，窃取次数与任务数同阶；每次偷 G 个，则窃取次数降为：

$$N_{\text{steal}} \approx \frac{M}{G}$$

- N_{steal}：总窃取次数；
- M：剩余任务总数；
- G：单次窃取的任务数。

窃取本身含通信成本，因此增大 G 同样是在压这项开销，代价与动态分配相同：偷得太多会重新造成不均。

[14] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

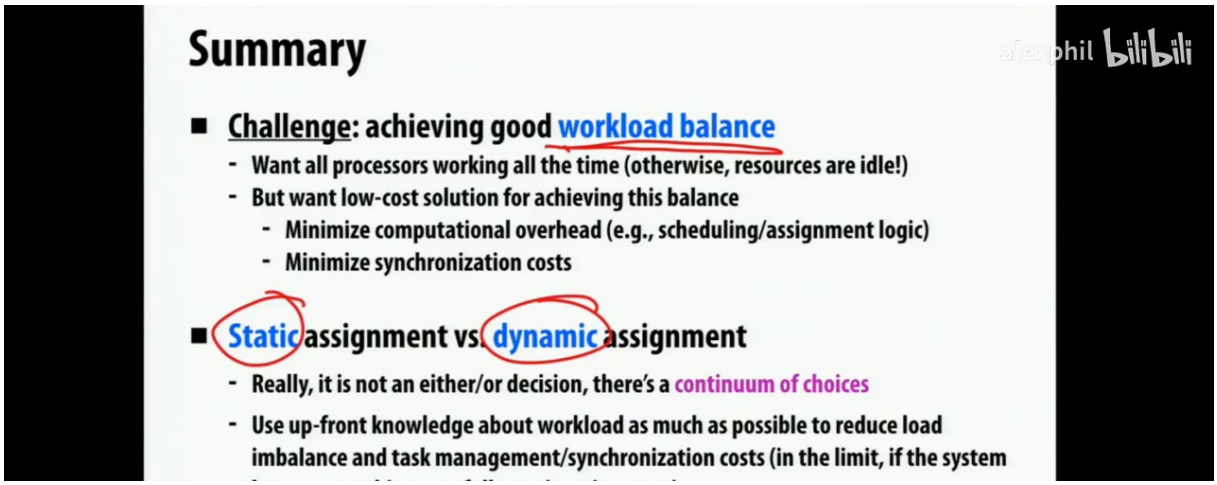


图 15: 负载均衡与低开销之间的权衡总结[15]

6.4 收尾阶段的集中争用

讲者也点出一个边界情况：真正到结束时，剩下的工作很少且都集中在一个队列里，此时争用会变得很严重。

但在通常情形下，一旦线程开始窃取，许多队列里往往还有不少工作可偷，所以实际做法是偷走其中一定比例。

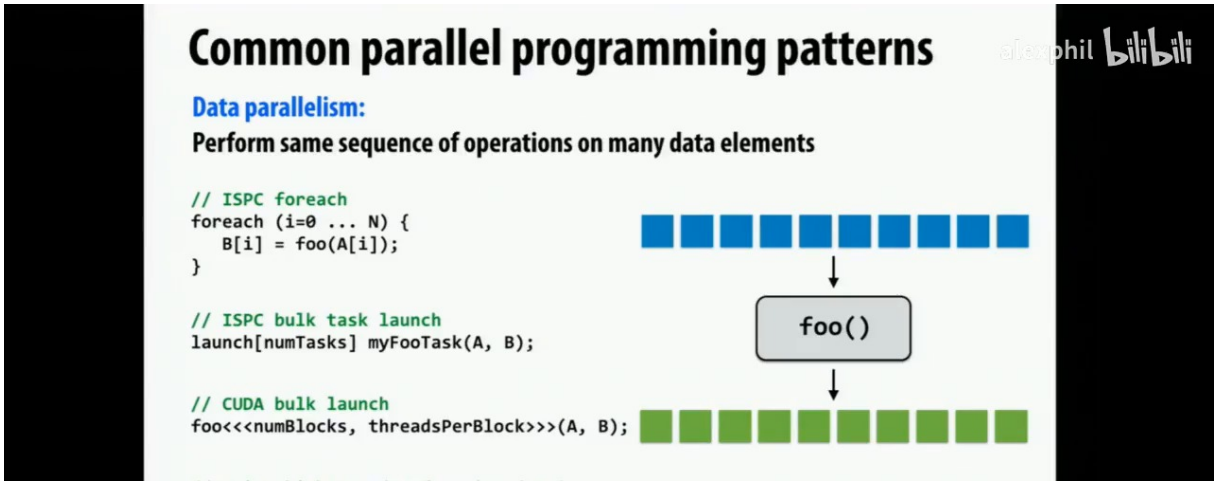


图 16: 常见并行编程模式：数据并行与 fork-join 等[16]

6.5 本章小结

分布式队列用私有性消除了中央队列的争用，work stealing 在必要时补上均衡。三个设计要点是：只在需要时窃取、随机选择目标、一次偷多个。它同时得到动态调度的均衡与静态调度般的局部性，代价是窃取发生时的通信与收尾阶段的集中争用。

讲者最后补充一个提醒：任务队列里的工作彼此不必独立，这一点在后续课程里会展开。

到这里分配与调度的原理已完整。剩下要看的是一套真实系统怎么把这些原理落实为语言原语。

[15] 视频画面时间区间：00:37:00–00:37:15。若为裁剪图，时间区间仍对应原视频画面。

[16] 视频画面时间区间：00:41:00–00:41:15。若为裁剪图，时间区间仍对应原视频画面。

7 Cilk 的 fork-join：抽象如何决定调度细节

讲者用 Cilk Plus 展示一个真实运行时系统的实现选择。

7.1 分治与 fork-join 模式

讲者先回顾常见并行模式，再聚焦到分治算法。

以快速排序为例：把数组分成两部分，各自递归排序，再合并。两部分互不依赖，这就是天然的并行来源。

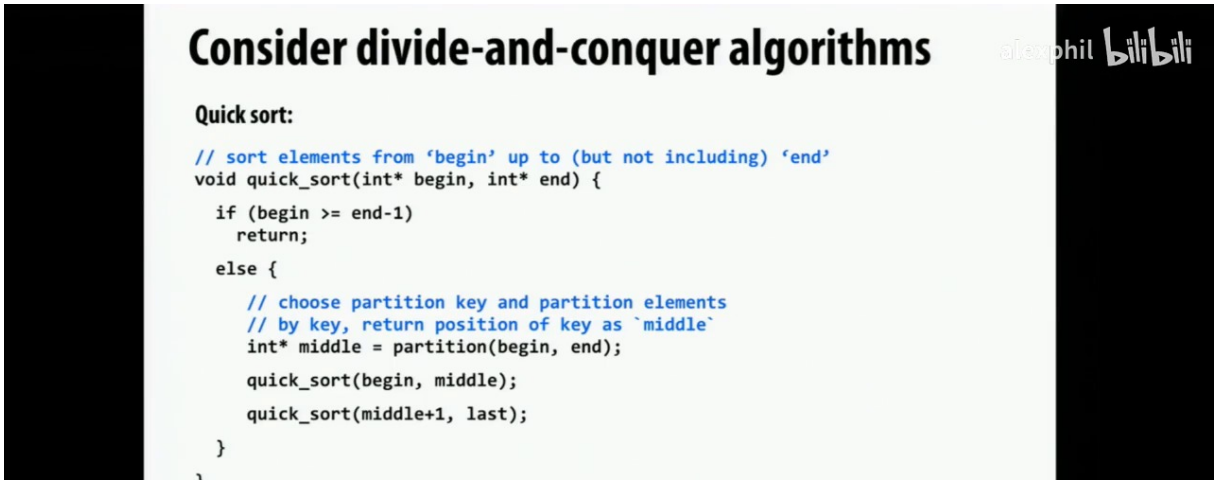


图 17: 分治算法：以快速排序为例[17]

表达这种结构的自然方式叫 fork-join：在一处 fork 出独立工作，在另一处 join 等它们完成。

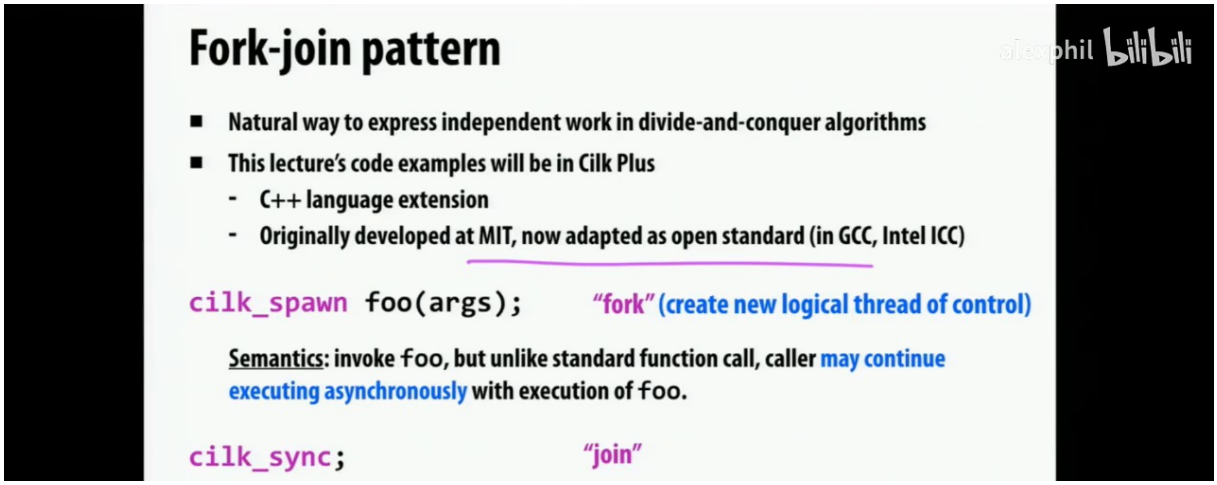


图 18: Fork-join 模式：表达分治算法中独立工作的自然方式[18]

[17] 视频画面时间区间：00:42:00–00:42:15。若为裁剪图，时间区间仍对应原视频画面。

[18] 视频画面时间区间：00:44:30–00:44:45。若为裁剪图，时间区间仍对应原视频画面。

7.2 spawn 与 sync

Cilk Plus 用两个关键字实现这个模式：cilk_spawn 表示这里可以并行，cilk_sync 表示等待所有 spawn 出来的工作完成。

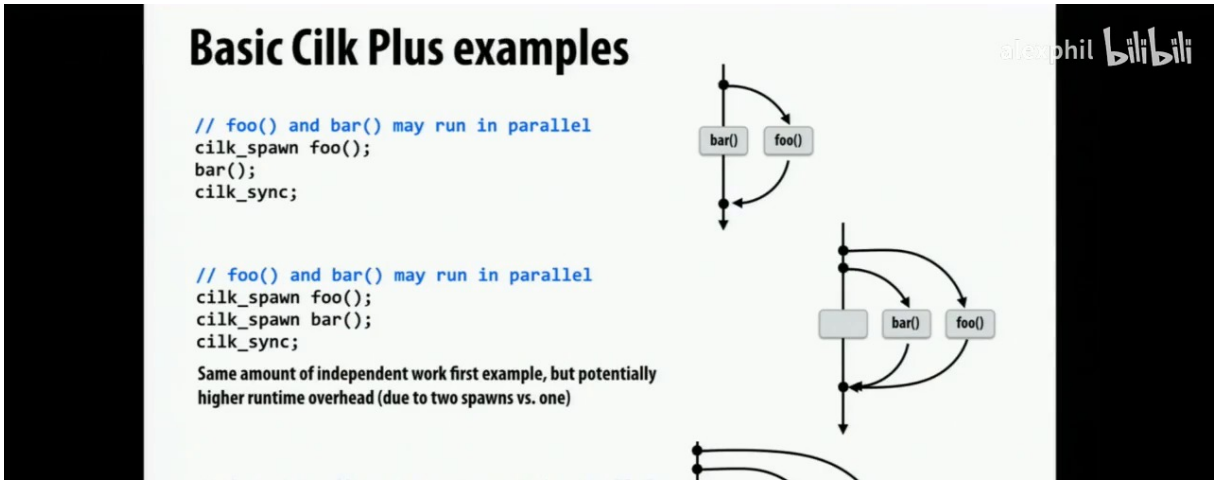


图 19: Cilk Plus 基础示例：cilk_spawn 与 cilk_sync[19]

关键在于讲者强调的一点：cilk_spawn 这个抽象并不指定哪个线程去跑哪一段。

抽象不承诺任何调度细节

cilk_spawn 只说这里可以并行，既不说在哪个核上跑，也不说谁先谁后，更不保证负载均衡。因此同一份 Cilk 程序在不同运行时、不同核数下的行为可以差别很大。这类抽象牺牲了可控性，换取了可移植性与简洁。

抽象只表达这¨ 里可以并行，¨ 具体怎么映射是实现的事。

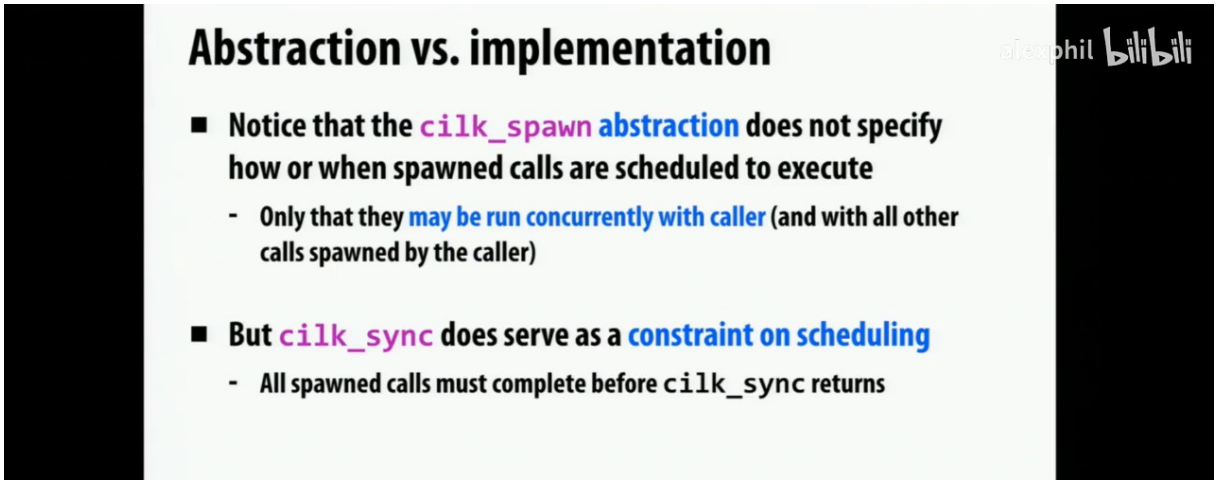


图 20: 抽象与实现：cilk_spawn 不指定线程，映射由运行时决定[20]

[19] 视频画面时间区间：00:46:30–00:46:45。若为裁剪图，时间区间仍对应原视频画面。
[20] 视频画面时间区间：00:48:00–00:48:15。若为裁剪图，时间区间仍对应原视频画面。

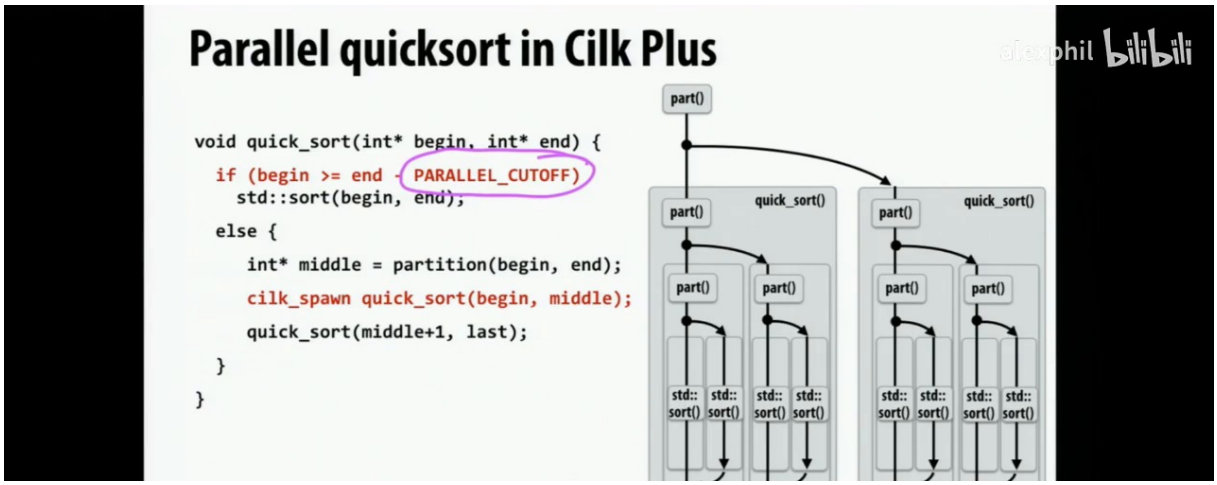


图 21: Cilk Plus 中的并行快速排序：递归部分用 spawn 并行[21]

7.3 为什么不能每个 spawn 开一个 pthread

讲者设想了最朴素的调度器：每个 cilk_spawn 就启动一个 pthread。这显然不可行——递归里会有成千上万个 spawn。

实际做法是运行时维护一个工作线程池。例如一台典型的笔记本可能四核，加上超线程就是八个硬件线程，Cilk Plus 就为你生成八个线程。讲者强调，重点不在于创建线程，而在于把线程指向该做的工作。

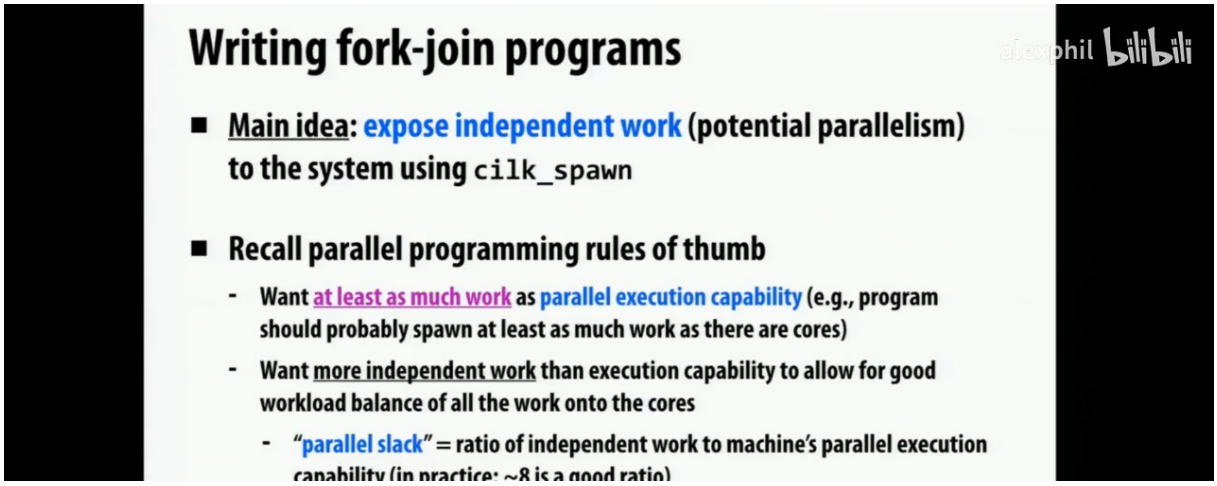


图 22: 编写 fork-join 程序：暴露独立工作并等待其完成[22]

[21] 视频画面时间区间：00:49:00–00:49:15。若为裁剪图，时间区间仍对应原视频画面。

[22] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

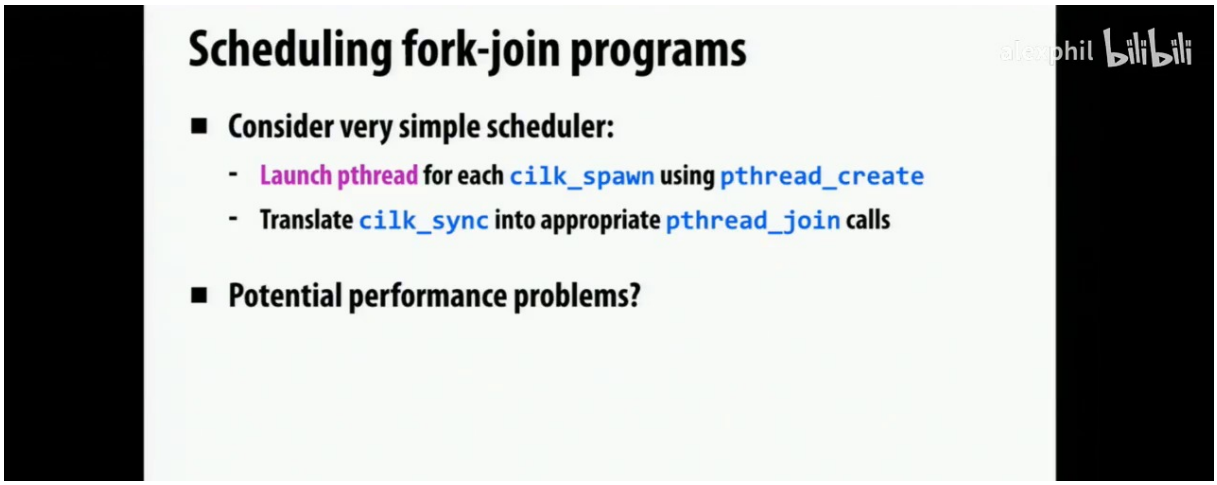


图 23: fork-join 程序的调度：最朴素的方案每 spawn 开一个线程[23]

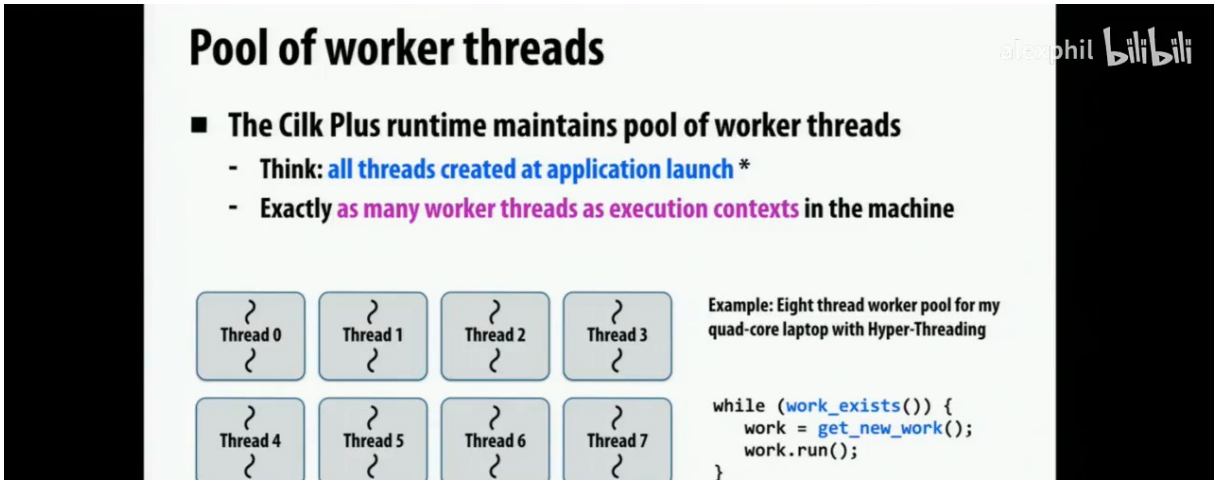


图 24: 运行时维护的工作线程池：所有线程共享同一批工作[24]

7.4 child 与 continuation

讲者引入两个术语来描述 spawn 处的两份可并行工作。被调用者 foo 叫 child（子任务）；调用之后要继续做的事叫 continuation（延续），在例子里就是 bar。

在 spawn 处，本线程要做其中一件，把另一件放进自己的队列，等待被别的线程偷走。

[23] 视频画面时间区间：00:52:30–00:52:45。若为裁剪图，时间区间仍对应原视频画面。

[24] 视频画面时间区间：00:54:30–00:54:45。若为裁剪图，时间区间仍对应原视频画面。

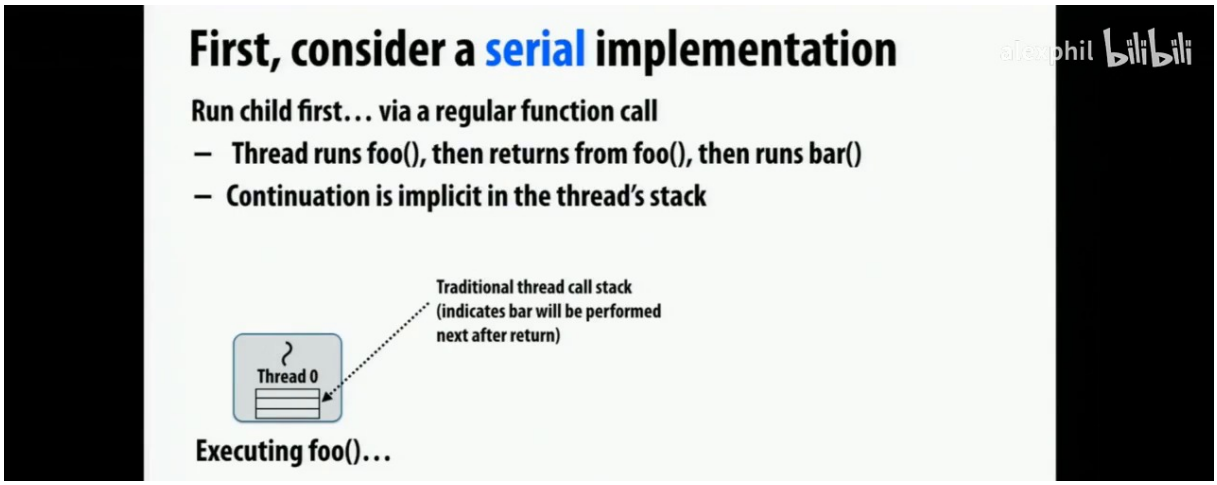


图 25: 先看串行实现：正常返回后继续执行后续代码[25]

讲者先看串行情形：没有 spawn 时，进入 foo、执行完返回，然后再做 bar。程序如何知道返回后要做什么，靠的是调用栈。

而在 Cilk 中，从 cilk_spawn 返回时不走普通返回路径，而是回到自己的队列取一件工作继续。这多了一点软件开销，但好处是若该线程随后空闲，它能发现自己的队列空了，并去别的队列偷工作。

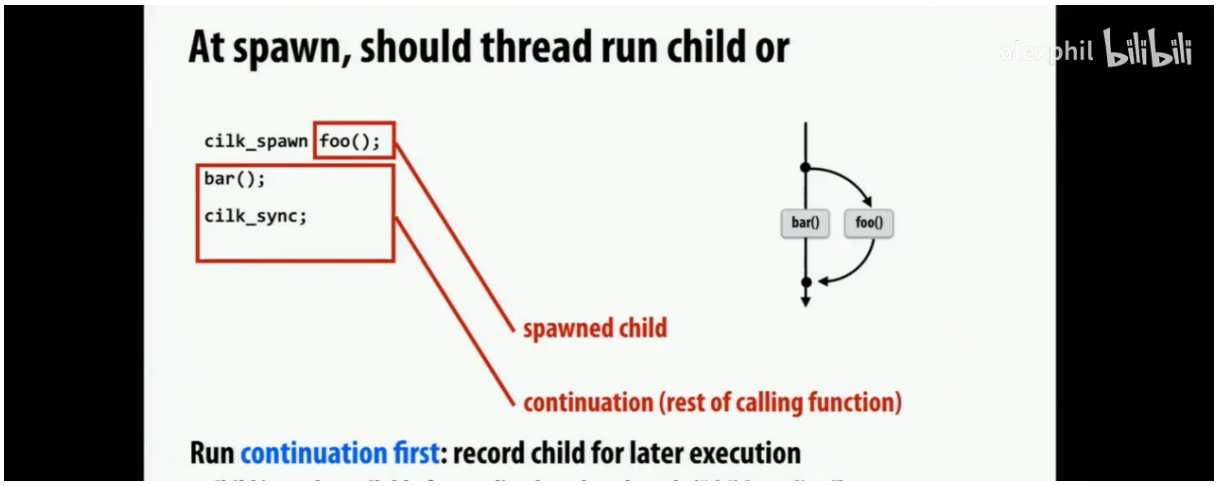


图 26: 每线程工作队列：遇到 spawn 时把 continuation 放进自己的队列[26]

[25] 视频画面时间区间：00:57:00–00:57:15。若为裁剪图，时间区间仍对应原视频画面。

[26] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。

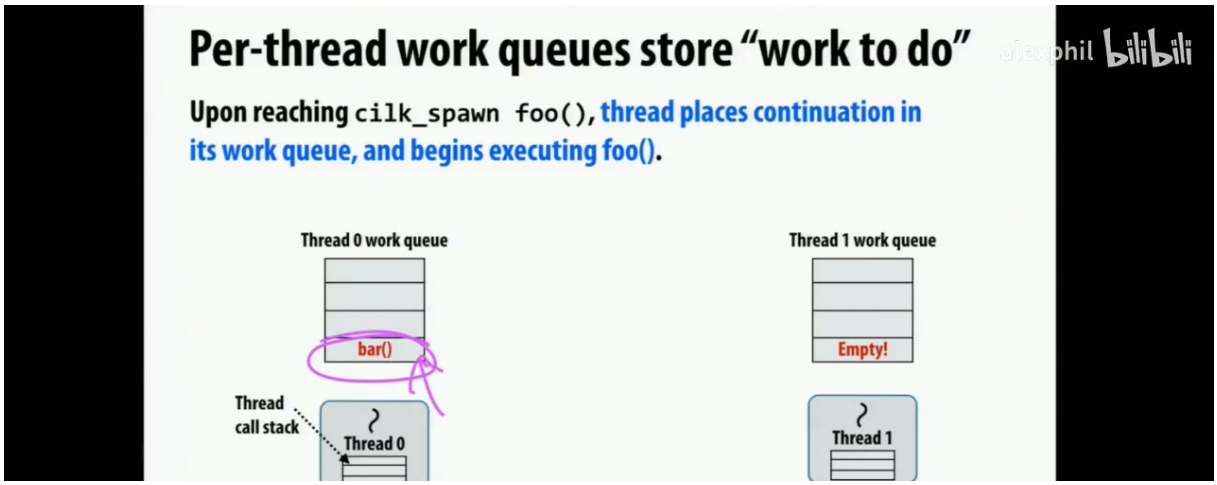


图 27: 每线程队列保存待办工作，空闲线程从别处窃取[27]

7.5 该先做 child 还是先做 continuation

这是本讲最有意思的权衡。学生提出可以直接把 foo 放进队列，自己去做 bar。讲者顺势展开讨论。

spawn 两种选择的表面优劣

先做 continuation：线程把大量后续工作堆进自己的队列，看起来储备充足，别的线程来偷也还有活干。先做 child：队列里只剩一件 continuation，一旦被别人偷走，本线程就没活可做了。仅凭这个观察，先做 continuation 似乎更安全。

讲者把这个情形推广成循环：对 n 个实例依次 spawn foo。若先做 continuation，每轮都会把 foo 放进队列然后继续，于是队列越堆越多。

若先做 child，线程执行 foo(0)，返回后发现刚过循环中那个点，于是继续下一轮执行 foo(1)。讲者指出这实际上很像深度优先遍历，访问方法的顺序很可能与顺序程序一致。

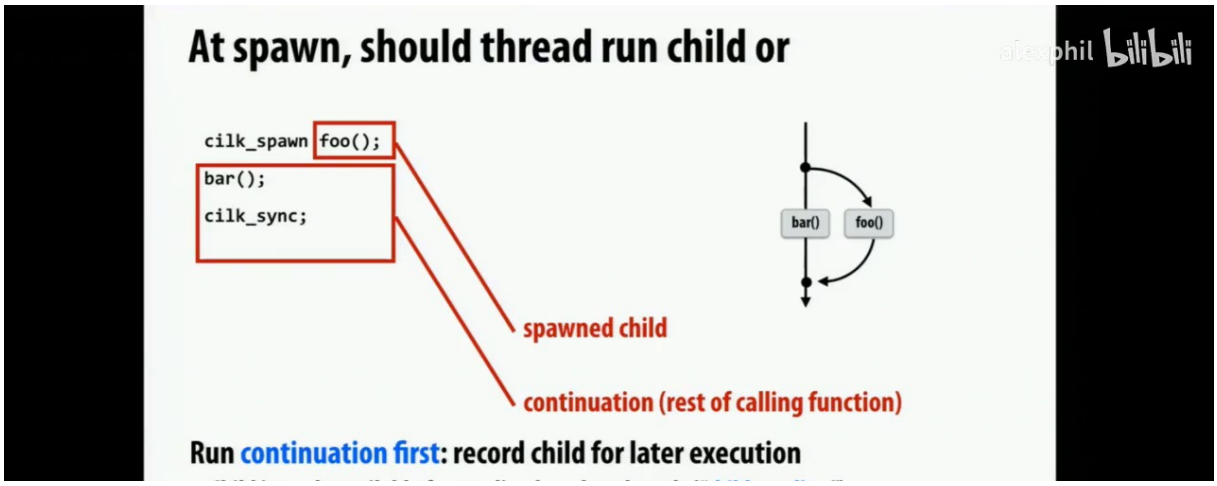


图 28: spawn 时该执行 child 还是 continuation[28]

[27] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

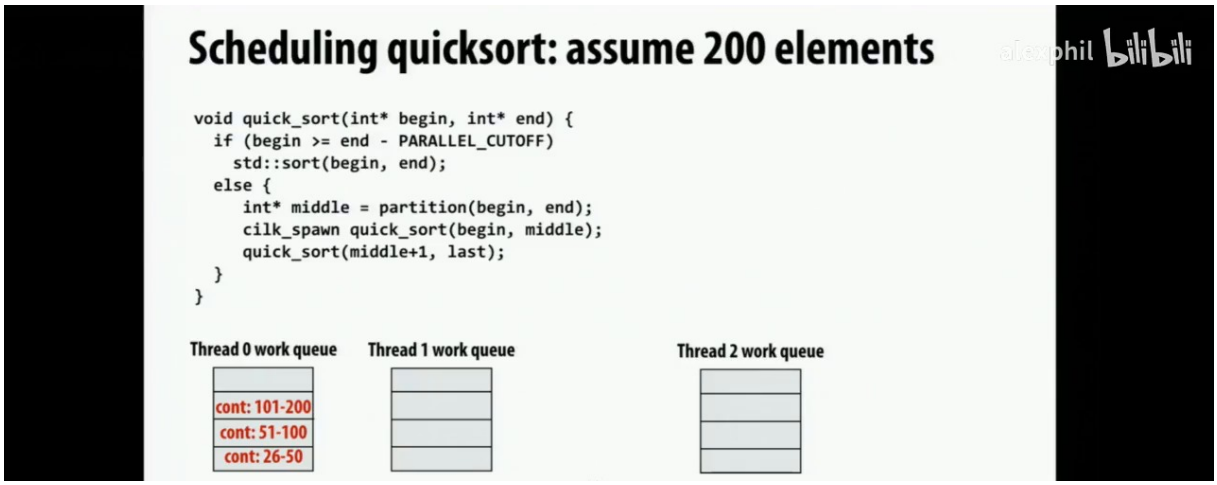


图 29: 快速排序调度示例：200 个元素时两种策略的差异[29]

7.6 Cilk 的真实选择

讲者让学生判断 Cilk 采用哪一种。答案是第二种——先做 child，尽管它表面上更不利。

理由与 work stealing 的局部性有关：先做 child 时，队列深度与递归深度成正比，被偷的任务与偷取者所需的数据在内存上也更接近，而先做 continuation 会制造出大量远端任务。此外讲者提到 Cilk 采用贪心策略：让最后完成的那份工作去执行同步，而不是固定某一方等待，这让实现更复杂但性能更好。

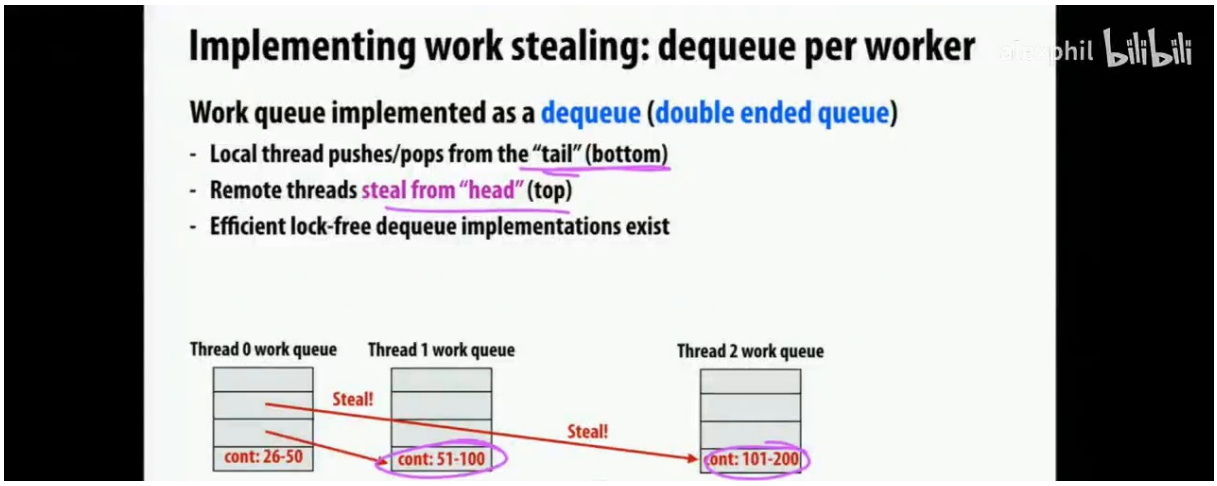


图 30: work stealing 的实现：每个 worker 一个双端队列[30]

[28] 视频画面时间区间：01:01:30–01:01:45。若为裁剪图，时间区间仍对应原视频画面。
[29] 视频画面时间区间：01:10:00–01:10:15。若为裁剪图，时间区间仍对应原视频画面。
[30] 视频画面时间区间：01:14:00–01:14:15。若为裁剪图，时间区间仍对应原视频画面。

双端队列让窃取与本地执行互不争用

work stealing 的工作队列通常实现为双端队列：本线程在自己一端压入与弹出（栈式，深度优先），窃取线程从另一端取走（队列式，取最老的任务）。两端操作不重叠，因此多数情况不需要加锁。

双端队列的两端分别服务两种角色

本线程从一端压入与弹出，行为像栈，因此走深度优先、数据局部性好；窃取者从另一端取走，行为像队列，因此拿到的是积压最久、通常也最大的任务。两端操作区间不重叠，多数情况下不需要加锁，这是 work stealing 低开销的根本原因。

这也解释了为什么先做 child（深度优先）能让窃取到的任务更大、更少。

7.7 本章小结

Cilk 用 spawn 与 sync 表达 fork-join，抽象不指定线程，运行时用线程池加分布式双端队列加 work stealing 实现。在 spawn 处它选择先执行 child，以获得深度优先的顺序与更好的局部性；表面上”队列永远不空”的 continuation-first 反而更差。

这一讲从”如何分工作”走到了”运行时如何调度”，全部结论都指向同一个结构：均衡与开销之间的取舍，以及用私有队列加按需窃取来同时争取两者。

8 总结与延伸

讲者的收尾很简短：我们看了 fork-join 并行，看到管理并行性上有一些有意思的权衡，目标是任务足够多但不能太多；也看了 Cilk 如何用 continuation、窃取与重新调度来做到这一点。

把全讲串起来：负载不均的代价按最慢者计算，20% 的差距就能浪费整个收尾阶段。静态分配零运行期开销，但依赖任务时间可预测。半静态在负载缓慢变化时用周期性重测取得折中。动态分配适应不可预测，但细粒度版本被锁竞争拖垮——那段等待本质上就是 Amdahl 定律中的串行时间。加大粒度能压住开销，但过大会重新引入不均，因此存在最优区间；理想策略是粒度随时间从大到小。分布式队列加 work stealing 同时取得均衡与局部性，三个设计要点是只按需窃取、随机选目标、一次偷多个。Cilk 则展示了如何在真实系统里落地这些选择，包括一个反直觉的结论：先执行 child。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 粒度是唯一同时影响均衡与开销的参数。加大粒度改善开销，加小粒度改善均衡，两者不可同时最优。所有调度器的默认值都在这条曲线上选点。
- work stealing 的局部性来自队列结构本身。私有队列加双端队列，使本线程与窃取者从两端操作，既避免争用，又让被偷的任务天然是较大的那批。
- Cilk 选择先做 child 是一次用局部性换表面均匀的取舍。它牺牲了”队列永远充实”的直觉优势，换取深度优先的顺序与近邻的数据。
- 收尾阶段的并行度衰减无法消除。任何分配策略最后都会只剩少量工作，改善的方向是让剩余任务更小，而不是追求全程满并行。

这堂课的边界也清楚。讲者反复表示队列中的工作彼此不必独立、通信与局部性的优化都会在后续课程展开。因此这里给出的是调度策略的选择框架，不含具体实现代码。

两个可以直接从本讲数据提出的下一步问题：其一，粒度既要足够小以均衡、又要足够大以省开销，那么在一台给定机器上，最优粒度是否与缓存容量相关？其二，work stealing 在收尾阶段会造成集中争用，这是否意味着应当让窃取本身也分级（先偷大块、再偷小块）？这两个问题都指向下一讲的通信与局部性。